

Seg Merge Split

线段树合并 & 分裂

线段树的合并与分裂是线段树的常用技巧，常见于权值线段树维护可重集的场景。

例如，树上某些结点处有若干操作，如果需要自下而上地将子节点信息传递给亲节点，而单个结点处的信息又方便用线段树维护时，就可以应用线段树合并的技巧控制整体的复杂度。

线段树合并

过程

顾名思义，线段树合并是指建立一棵新的线段树，这棵线段树的每个节点都是两棵原线段树对应节点合并后的结果。它常常被用于维护树上或是图上的信息。

显然，我们不可能真的每次建满一颗新的线段树，因此我们需要使用上文的动态开点线段树。

线段树合并的过程本质上相当暴力：

假设两颗线段树为 A 和 B，我们从 1 号节点开始递归合并。

递归到某个节点时，如果 A 树或者 B 树上的对应节点为空，直接返回另一个树上对应节点，这里运用了动态开点线段树的特性。

如果递归到叶子节点，我们合并两棵树上的对应节点。

最后，根据子节点更新当前节点并且返回。

▼ 线段树合并的复杂度

显然，对于两颗满的线段树，合并操作的复杂度是 $O(n)$ 的。但实际情况下使用的常常是权值线段树，总点数和 n 的规模相差并不大。并且合并时一般不会重复地合并某个线段树，所以我们最终增加的点数大致是 $O(\log n)$ 级别的。这样，总的复杂度就是 $O(n \log n)$ 级别的。当然，在一些情况下，可并堆可能是更好的选择。

实现

```
1 int merge(int a, int b, int l, int r) {
2     if (!a) return b;
3     if (!b) return a;
4     if (l == r) {
5         // do something...
6         return a;
7     }
8     int mid = (l + r) >> 1;
9     tr[a].l = merge(tr[a].l, tr[b].l, l, mid);
10    tr[a].r = merge(tr[a].r, tr[b].r, mid + 1, r);
11    pushup(a);
12    return a;
13 }
```

例题

▼ [luogu P4556 \[Vani 有约会\] 雨天的尾巴 / 【模板】线段树合并](#)

- 解题思路
- 参考代码

线段树分裂

过程

线段树分裂实质上是线段树合并的逆过程。线段树分裂只适用于有序的序列，无序的序列是没有意义的，常用在动态开点的权值线段树。

注意当分裂和合并都存在时，我们在合并的时候必须回收节点，以避免分裂时会可能出现节点重复占用的问题。

从一颗区间为 $[l, r]$ 的线段树中分裂出 $[s, t]$ ，建一颗新的树：

从 1 号结点开始递归分裂，当节点不存在或者代表的区间 $[l, r]$ 与 $[s, t]$ 没有交集时直接回溯。

当 $[l, r]$ 与 $[s, t]$ 有交集时需要开一个新结点。

当 $[l, r]$ 包含于 $[s, t]$ 时，需要将当前结点直接接到新的树下面，并把旧边断开。

▼ 线段树分裂的复杂度

可以发现被断开的边最多只会有 $\log n$ 条，所以最终每次分裂的时间复杂度就是 $O(\log n)$ ，相当于区间查询的复杂度。

实现

```
1 void split(int &p, int &q, int s, int t, int l, int r) {
2     if (t < l || r < s) return;
3     if (!p) return;
4     if (l <= s && t <= r) {
5         q = p;
6         p = 0;
7         return;
8     }
9     if (!q) q = New();
10    int m = (s + t) >> 1;
11    if (l <= m) split(ls[p], ls[q], s, m, l, r);
12    if (m < r) split(rs[p], rs[q], m + 1, t, l, r);
13    push_up(p);
14    push_up(q);
15 }
```

例题

▼ [P5494 【模板】线段树分裂](#)

- 解题思路
- 参考代码

习题

- [Luogu P4556 \[Vani 有约会\] 雨天的尾巴/【模板】线段树合并](#)
 - [Luogu P5494 【模板】线段树分裂](#)
 - [Luogu P1600 天天爱跑步](#)
 - [Luogu P4577 \[FJOI2018\] 领导集团问题](#)
 - [Luogu P2824 \[HEOI2016/TJOI2016\] 排序](#)
-

本页面最近更新：2025/4/7 00:24:17, [更新历史](#)

发现错误？想一起完善？ [在 GitHub 上编辑此页！](#)

本页面贡献者：[billchenchina](#), [c-forrest](#), [CCXXXI](#), [chenryang](#), [chenzheAya](#), [Chrogeek](#), [ChungZH](#), [CJSoft](#), [cjsoft](#), [countercurrent-time](#), [DawnMagnet](#), [Early0v0](#), [Enter-tainer](#), [ethan-enhe](#), [GavinZhengOI](#), [Haohu Shen](#), [Henry-ZHR](#), [HeRaNO](#), [hjsjhn](#), [hly1204](#), [hsfzLZH1](#), [iamtwz](#), [lr1d](#), [jaxvanyang](#), [Jebearssica](#), [kenlig](#), [konnyakuxzy](#), [ksyx](#), [luoguojie](#), [Marcythm](#), [megakite](#), [Menci](#), [moon-dim](#), [NachtgeistW](#), [onelittlechildawa](#), [orzAtalod](#), [ouuan](#), [shadowice1984](#), [shawlleyw](#), [shuzhouliu](#), [StudyingFather](#), [SukkaW](#), [Tiphereth-A](#), [wy-luke](#), [x2e6](#), [Xeonacid](#), [Ycrpro](#), [yifan0305](#), [zeningc](#)

本页面的全部内容在 [CC BY-SA 4.0](#) 和 [SATA](#) 协议之条款下提供，附加条款亦可能应用